

AUTOMATED WALL DAMAGE DETECTION AND REPAIR ESTIMATION SYSTEM

A Technical Research Report

Computer Vision • AI • Backend Systems

Prepared by

Wall Damage AI Backend

AUTOMATED WALL DAMAGE DETECTION AND REPAIR ESTIMATION SYSTEM: A TECHNICAL DEEP DIVE

EXECUTIVE SUMMARY

The maintenance of physical infrastructure, specifically vertical load-bearing structures (walls), is a critical component of building management. Traditional inspection methods are labor-intensive, subjective, and prone to human error. This report details the technical architecture and implementation of a hybrid Artificial Intelligence system designed to automate the detection, classification, and remediation planning for wall damages.

The system leverages a microservices-inspired architecture, utilizing **NestJS** for robust implementation of the API Gateway and **Python** for high-performance computational inference. At its core, the system employs an **Ensemble Learning Strategy**, combining the localization capabilities of **YOLOv8** (You Only Look Once) with the global classification power of **ResNet-50** (Residual Networks). This dual-model approach is governed by a custom "Voter Algorithm" to ensure high-fidelity results. Furthermore, the system integrates **Generative AI (Google Gemini 1.5 Pro)** to provide context-aware, step-by-step repair instructions, effectively bridging the gap between defect detection and actionable maintenance advice.

1. INTRODUCTION

1.1 Problem Statement

In civil engineering and facility management, "Wall Damage" is a broad category encompassing various defects such as cracking, paint flaking, material loss, and water intrusion.

- Cracks**: Can indicate structural failure or settling.
- Peeling/Flaking Paint**: Often aesthetic but can signify moisture issues.
- Missing Pieces**: Physical impact damage requiring filling.
- Water Damage**: Critical indicators of plumbing or roof failures.

Manual identification of these issues requires trained personnel to visit sites, document findings, and draft reports. This process is:

- Slow**: Turnaround time can be days or weeks.
- Expensive**: High labor costs for site visits.
- Inconsistent**: Different inspectors may classify the same damage differently.

1.2 Proposed Solution

The Wall Damage Backend acts as a centralized processing unit that accepts digital imagery of walls and outputs a comprehensive forensic report. It democratizes structural inspection, allowing any user with a smartphone to perform a preliminary assessment that is backed by state-of-the-art computer vision models.

2. SYSTEM ARCHITECTURE

The system is built on a **Layered Hybrid Architecture**, enabling the separation of concerns between web-facing interactions and computationally intensive AI tasks.

2.1 High-Level Component Design

```
graph TD
    Client[Client App (Mobile/Web)] -->|HTTPS POST /damage/analyze| Server[NestJS Server (Node.js)]

    subgraph "Application Layer (NestJS)"
        Controller[AppController] -->|File Upload| Multer[Multer Middleware]
        Multer -->|Save| Disk[Uploads Directory]
        Controller -->|Pass Path| Service[AppService]
    end

    subgraph "Bridge Layer"
        Service -->|Spawn Process| PyShell[Python-Shell]
        PyShell -->|STDIN| PythonRuntime[Python 3.x Environment]
    end

    subgraph "Intelligence Layer (Python)"
        PythonRuntime -->|Load| Script[damage_analyzer.py]
        Script -->|Inference| YOLO[YOLOv8 Model]
        Script -->|Inference| ResNet[ResNet-50 Model]

        YOLO -->|BBBox Data| Logic[Voting Algorithm]
        ResNet -->|Class Probs| Logic

        Logic -->|Selected Class| GenAI[GenAI Module]
        GenAI -->|API Call| Gemini[Google Gemini API]
    end

    Gemini -->|JSON Repair Plan| GenAI
    GenAI -->|Enriched Data| Script
    Script -->|JSON Output| PyShell
    PyShell -->|Promise Resolve| Service
    Service -->|HTTP 200| Client
```

2.2 The Application Layer (NestJS)

- **NestJS** is chosen for the application layer due to its modularity and extensive support for TypeScript. It serves as the orchestrator.
- **Request Handling**: The `AppController` (`src/app.controller.ts`) listens on the `/damage/analyze` route. It employs the `@UseInterceptors(FileInterceptor('image'))` decorator to offload stream

handling to `Multer`.

- **Input Validation**: A custom filter ensures only valid image formats (jpg, png, bmp) are processed, preventing malicious file uploads that could exploit the image processing libraries.
- **Storage Strategy**: files are temporarily stored in the `uploads/` directory with unique timestamps. This "Store-then-Process" approach decouples the upload speed from the processing speed.

2.3 The Bridge Layer (Inter-Process Communication)

A critical architectural decision was to avoid running Python web frameworks (like Flask/FastAPI). hosting the AI inside a separate service would add network latency (HTTP overhead between Node and Python).

Instead, we use **Python-Shell**.

- **Mechanism**: Node.js spawns a child process invoking the python executable directly.
- **Communication**: Data is passed via `stdin` (command line arguments for file paths) and retrieved via `stdout` (standard output).
- **Benefits**:
 - **Low Latency**: No TCP handshake overhead.
 - **Environment Isolation**: The Python environment is a self-contained Conda environment, preventing dependency conflicts with the host system.

3. ARTIFICIAL INTELLIGENCE METHODOLOGY

The intelligence core relies on a **Multi-Model Voting System**. Single models often suffer from bias or specific failure modes. By combining two distinct architectures, we achieve a robust "Expert Opinion" system.

3.1 Model 1: YOLOv8 (You Only Look Once)

- **Role**: Object Detection & Localization
- **File**: `models/best.pt`

YOLOv8 is a state-of-the-art object detection model known for its speed and accuracy. Unlike sliding-window approaches, YOLO treats detection as a single regression problem.

- **Why it's used here**: Wall damage is often localized. A large wall might have a small crack. A classifier looking at the whole wall might miss it. YOLO divides the image into a grid and predicts bounding boxes.
- **Output**:
- **Class**: What is the damage? (e.g., `crack\_damages`)
- **Confidence**: How sure is it? (0.0 - 1.0)
- **Bounding Box**: coordinates `(x1, y1, x2, y2)`. This is crucial for **Area Estimation**. The system calculates the area of the box relative to the image to estimate the extent of the damage (sq. meters).

3.2 Model 2: ResNet-50 (Residual Network)

- ***Role***: Global Image Classification
- ***File***: `models/resnet50\_classifier\_final.h5`

ResNet-50 is a deep Convolutional Neural Network (CNN) that introduced "skip connections" to allow training of very deep networks without vanishing gradients.

- ****Why it's used here****: Textures matter. Flaking paint looks different from water damage, even if the "shape" is undefined. ResNet looks at the **entire** image context (color histograms, texture gradients).
- ****Output****: A probability vector summing to 1.0 across 4 classes:
 1. `crack\_damages`
 2. `flaking\_paint`
 3. `missing\_piece`
 4. `water\_damage`

3.3 The Core Logic: Hybrid Decision Algorithm

The `damage\_analyzer.py` script implements a sophisticated decision tree to reconcile differences between YOLO and ResNet. This is the "Brain" of the operation.

- ***The Logic Flow (Code Analysis):***
 1. ****Execution****:
 - ResNet runs on the resized image (224x224).
 - YOLO runs on the original resolution image.
 2. ****The Prioritization Rule****:
 - ****Condition A (High Confidence YOLO)****: If YOLO detects a damage object with ****> 50% confidence****, it wins.
 - ***Rationale***: If a specific object (crack) is found, it is almost certainly there. Global classifiers are easily confused by background noise (wallpaper patterns, shadows).
 - ****Condition B (Low Confidence YOLO vs. Low Confidence ResNet)****: If YOLO detects something (even < 50%) but ResNet is **also** unsure (< 50%), YOLO beats ResNet.
 - ***Rationale***: A weak specific detection is better than a weak global guess.
 - ****Condition C (ResNet Fallback)****: If YOLO finds **nothing** (empty list), we default to ResNet's highest probability class.
 - ***Rationale***: Sometimes damage covers the whole wall (like widespread water stains) and YOLO can't find a distinct "edge" or "box". ResNet excels here.

4. GENERATIVE AI INTEGRATION (GEMINI 1.5)

To move beyond simple "labelling" to "consulting", the system integrates a Large Language Model (LLM).

4.1 The Prompt Engineering Strategy

The script ``LLM_explanations.py`` constructs a dynamic prompt. It does not just ask "What is this?". It injects the **proven** diagnosis from the previous step.

- ***Prompt Template***:

> "Explain why ``{damage_type}`` occurs on walls, how to resolve it, and list steps and materials needed in a structured JSON format..."

- ***Key Constraints***:

1. ***Context Injection***: The variable ``{damage_type}`` serves as ground truth. This prevents the LLM from hallucinating a different damage type.
2. ***Format Enforcement***: "Ensure the output is ONLY the JSON object". This is critical because the Python script needs to parse the response programmatically.
3. ***Role Definition***: Implicitly, the LLM is acting as a "Structural Engineer".

4.2 Fallback Mechanisms (The "Static Dictionary")

Reliability is paramount. APIs fail, networks timeout, and quotas expire. The system includes a hardcoded dictionary (``EXPLANATIONS`` in ``damage_analyzer.py``) containing expert-written generic advice for all 4 categories.

- ***Trigger***: If the ``subprocess.run`` calling the LLM script returns a non-zero exit code or throws a JSON parse error.
- ***Benefit***: The user **always** gets a report, even if the "AI Consultant" is offline.

5. FORENSIC LOG ANALYSIS

The ``damage_analyzer.log`` file provides a black-box flight recorder for the AI's decision process. Analyzing this log reveals the effectiveness of the Hybrid Logic.

Case Study 1: The "Easy" Case

```
INFO - ResNet-50 probabilities: [[0.24, 0.25, 0.25, 0.25]]
INFO - YOLOv8 damage_type: flaking_paint, confidence: 0.85
INFO - Selected damage_type: flaking_paint
```

- ***Analysis***: ResNet was completely confused (near uniform distribution of 0.25). YOLO was very confident (0.85).
- ***Result***: The system correctly prioritized YOLO, saving the user from a "random guess" via ResNet.

Case Study 2: The "Subtle" Case

```
INFO - ResNet-50 probabilities: [[0.60, 0.10, 0.15, 0.15]]
```

INFO - YOLOv8 damage_type: None, confidence: 0.0
INFO - Selected damage_type: crack_damages (from ResNet)

- **Analysis**: YOLO failed to find a bounding box (perhaps the crack was hairline and invisible to the object detector). ResNet, seeing the texture anomaly, identified it as a crack with 60% confidence.
- **Result**: The fallback mechanism worked. Without ResNet, this would have returned "No Damage Found".

Case Study 3: The "Conflict"

INFO - ResNet-50 damage_type: flaking_paint, confidence: 0.55
INFO - YOLOv8 damage_type: crack_damages, confidence: 0.65
INFO - Selected damage_type: crack_damages

- **Analysis**: Both models are fairly confident but disagree.
- **Result**: The logic dictates YOLO wins. This is generally the correct behavior in computer vision; object detectors have lower false-positive rates for specific shapes than global classifiers processing complex backgrounds.

6. BACKEND IMPLEMENTATION DETAILS

6.1 Dependency Injection

NestJS utilizes a powerful DI container. `AppService` is injected into `AppController`. This allows for easy unit testing—we could mock `AppService` to return fake formatted data without actually spawning Python processes during test runs.

6.2 Asynchronous Processing

The `analyzeDamage` method in `AppService` wraps the `PythonShell.run` interaction in a `Promise`.

```
return new Promise((resolve, reject) => {  
  PythonShell.run(...)  
    .then(results => resolve(parse(results)))  
    .catch(err => reject(err));  
});
```

This ensures the Node.js event loop is **never blocked**. The server can continue identifying other requests while waiting for the Python script (which takes 2-5 seconds) to complete.

6.3 JSON Parsing Resilience

The output from Python includes logs (INFO lines) mixed with the final JSON result. The parser in `AppService.ts` is robust:

1. It iterates through *every line* of output.
2. It attempts to `JSON.parse()` each line individually.
3. It checks for the "signature" keys (`damage_type` inside the object).
4. It ignores all other lines (logging noise).

This allows developers to add ``print("Debug info")`` to the Python script without breaking the API response, significantly aiding development velocity.

7. CONCLUSION

The Wall Damage Backend represents a mature implementation of Applied AI. It does not simply "wrap" a model; it engineers a solution around the models' strengths and weaknesses.

1. **Robustness**: Through the Hybrid Voting Logic.
2. **Utility**: Through the GenAI repair instructions.
3. **Scalability**: Through the async, event-driven Node.js architecture.

This system effectively reduces the barrier to entry for structural assessment, providing professional-grade insights at zero marginal cost per inspection.